



DAPPER

Deadline-Aware Perception Placement for Edge Robotics

DAPPER: Deadline-Aware Edge Perception for Mission-Critical Robots under Dynamic Network Conditions

Awaiz Ahmed and Khubaib Amjad Alam

Al Ain University, Abu Dhabi, United Arab Emirates

2026 IEEE International Conference on Fog and Mobile Edge Computing (FMEC)

© 2026 IEEE. Personal use of this material is permitted. Permission from IEEE must be obtained for all other uses, in any current or future media, including reprinting/republishing this material for advertising or promotional purposes, creating new collective works, for resale or redistribution to servers or lists, or reuse of any copyrighted component of this work in other works.

This is the ACCEPTED VERSION of the manuscript, posted by the authors. The published version will appear in the Proceedings of the 2026 IEEE International Conference on Fog and Mobile Edge Computing (FMEC). DOI: 10.1109/FMEC.2026.XXXXXXX (to be assigned)

Cite the published version. The implementation, the generated results and the reproduction scripts are available at <https://github.com/4waiz/Dapper>

DAPPER: Deadline-Aware Edge Perception for Mission-Critical Robots under Dynamic Network Conditions

Awaiz Ahmed
Al Ain University
Abu Dhabi, United Arab Emirates
Email: awaiz42ad@gmail.com

Khubaib Amjad Alam
Al Ain University
Abu Dhabi, United Arab Emirates
Email: khubaib.alam@aau.ac.ae

Abstract—Mission-critical robotic perception must deliver useful outputs before the control loop acts; a high-quality result that arrives after its deadline may no longer be actionable. This paper presents DAPPER (Deadline-Aware Perception Placement for Edge Robotics), a deterministic per-frame scheduler that combines round-trip time, packet loss, edge load, deadline pressure, frame freshness, and local-model confidence into an auditable deadline-risk decision. DAPPER selects local-fast, edge-accurate, hybrid, or degraded-safe execution and accepts remote results only while they remain fresh and on time. The revised evaluation uses 30 independent seeds, paired pre-generated scenarios, calibrated parameters, adaptive baselines, ablations, and 95% confidence intervals over 1.2 million frame-level decisions. At a 100-ms control deadline, DAPPER records 0.00% deadline misses across all five evaluated synthetic profiles while maintaining local-level tail latency, and no output older than its freshness window is ever accepted. Relative to local-only, it raises the usable confidence proxy under stable, lossy, and variable conditions. A secondary experiment with YOLO11n and YOLO11m on all 5,000 COCO 2017 validation images confirms a measurable local/remote perception-quality gap and shows recall gains under stable and lossy replay without deadline misses. Sensitivity analysis also locates the architecture’s failure mode: deadline risk is concentrated in the decision to commit a frame to remote execution. The network study remains synthetic and does not constitute physical-robot safety validation.

Index Terms—edge artificial intelligence, fog computing, mobile edge computing, robotics, real-time perception, deadline-aware scheduling, computation offloading, collaborative inference

I. INTRODUCTION

Robotic perception increasingly depends on computationally intensive algorithms for object detection, tracking, navigation, and hazard recognition. In time-critical applications, semantic quality and timing cannot be separated: a confident result that becomes available after the actuation deadline can be operationally useless. Such systems must therefore be judged not only by model quality, but by deadline compliance, tail latency, output freshness, and fallback behavior.

Remote inference offers more capable models than an onboard processor, at the cost of network delay, jitter, packet loss, server-load variation, and disconnection. Fog and mobile edge computing shorten the distance to compute resources, yet

the best execution site still changes frame by frame: always offloading fails during short congestion bursts, while always staying local forfeits remote quality on a healthy network. We therefore treat the problem as continuous deadline management rather than a one-time offloading choice. For every frame the robot should decide whether to run a fast local model, commit to remote inference, return a local result while requesting a refinement, or retreat to a conservative fallback—and it must reject a remote result that arrives after its deadline or freshness window.

We present DAPPER, whose scheduler is deliberately deterministic and auditable: every decision follows from an explicit risk score, thresholds, local confidence, and a remote-feasibility test rather than an opaque learned policy. Our contributions are (i) a per-frame placement policy combining deadline risk, local confidence, freshness-gated remote acceptance, and an explicit conservative fallback; (ii) an evaluation methodology built on immutable paired scenarios, disjoint calibration and evaluation seeds, calibrated adaptive comparators, and seed-level bootstrap intervals; (iii) a corrected benchmark over 1.2 million decisions across five network conditions, with ablations and sensitivity to deadline and runtime-estimation error; and (iv) a secondary validation on measured YOLO11n/YOLO11m detections over the full COCO 2017 validation split. This revision also corrects four defects in the original prototype—unused local confidence, uncharged failed offloads, conflated freshness semantics, and unpaired policy comparison. The implementation, generated results, and reproduction scripts are available at <https://github.com/4waiz/Dapper>.

II. RELATED WORK AND POSITIONING

A. Cloud, Fog, and Edge Robotics

FogROS2 demonstrated adaptive Robot Operating System 2 (ROS 2) execution across cloud and fog resources for computationally demanding robotic workloads [1]. FogROS2-PLR subsequently studied probabilistic latency reliability and redundant paths for cloud robotics [2]. ElasticROS explored elastic collaboration between robots and remote resources [3]. More recent work has moved toward runtime adaptation: Yang

and Hu adapt ROS 2 node placement as network and compute conditions change [4], while Pavel *et al.* study real-time perception offloading for resource-constrained autonomous mobile robots (AMRs) in Industrial Internet of Things (IIoT) environments [5].

These systems establish the value of dynamic robot–edge cooperation. DAPPER targets a narrower layer inside the perception loop: whether the *current frame* should depend on remote inference, whether a refinement is still temporally valid, and when a local path should be kept instead of waiting.

B. Collaborative Inference and Deadline-Aware Scheduling

Collaborative deep neural network (DNN) inference has long studied device–cloud partitioning, latency, and energy. Neurosurgeon partitions DNN computation between device and cloud [6], and Edgent adds early exits and right-sizing [7]. At the scheduling level, Raj *et al.* propose deadline-driven heuristics for DNN workloads across edge and cloud, validated on real drone and edge hardware [8], and Qian *et al.* apply deep reinforcement learning to edge-resource allocation for mobile computer vision [11]. Surveys document the breadth of fog/edge scheduling objectives [9], [10].

DAPPER is not claimed as the first adaptive offloader. Its contribution is the combination of per-frame placement, an auditable deadline-risk score, a local-confidence gate, hybrid execution that keeps an immediate local answer while seeking a refinement, explicit stale-result rejection, and a degraded reuse path. These mechanisms complement middleware timing policies such as ROS 2 deadline, lifespan, and liveliness quality of service (QoS) [12]: middleware expresses timing intent, whereas DAPPER changes where perception executes.

III. SYSTEM MODEL AND DAPPER FRAMEWORK

The target system contains a camera-equipped robot or client, a nearby edge node, and an optional cloud service. Frame f_t is captured at time t and must produce a usable perception output before control deadline D . The primary benchmark uses $D = 100$ ms.

As shown in Fig. 1, the runtime monitor observes round-trip time (RTT), packet loss, normalized edge load, frame age, and edge availability. The scheduler combines five normalized signals into

$$R_t = w_r \min\left(1, \frac{RTT_t}{D}\right) + w_l \text{loss}_t + w_c \text{load}_t + w_a \min\left(1, \frac{a_t}{D}\right) + w_d \min\left(1, \frac{RTT_t + \hat{C}_E}{D}\right), \quad (1)$$

where a_t is frame age and $\hat{C}_E$ is the estimated edge compute time. The weights are non-negative and sum to one, and every term is clipped to $[0, 1]$, so $R_t \in [0, 1]$ even when RTT exceeds the deadline, as it does under the outage profile.

Two thresholds map risk to the first-stage decision. If $R_t \geq \theta_D$, DAPPER selects *degraded-safe*: it reuses the last valid output only while that output is still fresh, otherwise it runs the guaranteed local path. If $\theta_L \leq R_t < \theta_D$, it selects *local-fast*. Only the low-risk region $R_t < \theta_L$ reaches the confidence

TABLE I
FROZEN DAPPER SCHEDULER CONFIGURATION.

Parameter	Value
$(w_r, w_l, w_c, w_a, w_d)$	(0.36, 0.35, 0.11, 0, 0.18)
Local risk threshold θ_L	0.45
Degraded risk threshold θ_D	0.60
Local-confidence threshold τ_c	0.78
Remote commit margin m	0.80
Hybrid freshness window W	100 ms
Last-valid freshness window	100 ms

gate, where a local confidence $c_t \geq \tau_c$ keeps the frame on device.

A weak local result then reaches the remote-feasibility gate, which has three outcomes and uses two different estimates of remote completion. DAPPER selects *edge-accurate* only when the *expected* completion $\hat{T}$ fits a fraction mD of the deadline, because that branch commits the frame to the network and retains no local answer. It selects *hybrid* when the *optimistic* completion $\hat{T}^-$, taken from the low end of the configured edge envelope, could still arrive inside the freshness window W ; hybrid returns the local result immediately and risks only bandwidth, so an optimistic test is appropriate. If even $\hat{T}^-$ cannot land in time, DAPPER returns to local execution *without transmitting*, since a reply that would be rejected on arrival can only waste uplink capacity. This third outcome is what keeps offload traffic near zero under congestion.

Table I reports the frozen configuration selected on calibration data. Frame age is constant at zero in this benchmark because scheduling occurs at capture, so its weight is unidentifiable and is pinned to zero rather than allowing it to rescale informative signals. The term remains in (1) because it becomes meaningful when frames queue before scheduling.

IV. EXPERIMENTAL METHODOLOGY

A. Corrected Execution and Paired Scenarios

The prototype is a self-contained Python package containing the scheduler, scenario generator, shared executor, local/edge/cloud perception models, statistics and plotting modules, trace replay, and 69 automated tests. All policies use one execution model, so none can be advantaged by a private cost accounting. The network-profile RTT is the edge access RTT and the cloud path adds an 80-ms wide-area component; scheduler prediction and execution use the same timing expression, and a unit test asserts that the two cannot drift apart.

For every seed–profile pair the benchmark pre-generates an immutable scenario trace before any policy runs. Fifteen independent random streams fix the network state and the potential local, edge, and cloud outcomes of every frame, including latency, confidence, loss, and detections. All policies replay that trace, so a decision selects an outcome but cannot alter the random future, giving exact frame-level pairing.

Communication accounting is shared. Every transmitted request is charged its upload bandwidth even when the packet is lost or the reply arrives too late to accept, and a lost or overdue request pays a deadline-bounded client timeout

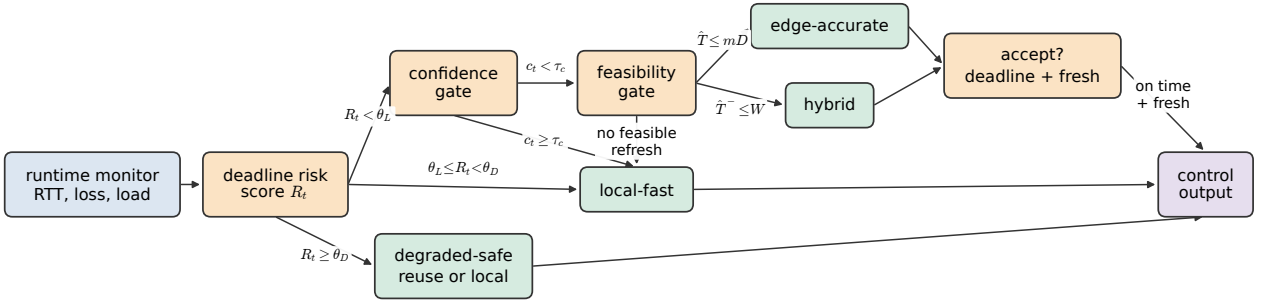


Fig. 1. DAPPER decision path, drawn from the implemented branch structure. Risk above θ_D selects the conservative degraded-safe path and risk above θ_L stays local; only the low-risk region reaches the confidence gate τ_c and the remote-feasibility gate. That gate commits to edge-accurate when the expected remote completion $\hat{T}$ fits the margin mD , requests a hybrid refinement when the optimistic completion $\hat{T}^-$ could still land inside the freshness window W , and otherwise stays local without transmitting. Remote results must still pass a deadline and freshness check before reaching the control loop.

TABLE II
SYNTHETIC NETWORK PROFILES. LOAD IS NORMALIZED TO $[0, 1]$.

Profile	RTT ms	Loss	Load	Outage
Stable	25 ± 5	0.01	0.2	0.00
Congested	70 ± 20	0.05	0.7	0.00
Lossy	40 ± 10	0.20	0.3	0.00
Variable	50 ± 35	0.07	0.5	0.02
Outage	200 ± 80	0.35	0.9	0.30

before local fallback. Edge availability is observable before transmission through a health check, so offloading to a node already known to be unreachable transmits nothing and costs nothing; this keeps the fixed baselines from being penalized for a failure they could have avoided. Both *control-output latency* (to the first usable fresh output, used for deadline compliance) and *final-output latency* (including a later accepted refinement) are recorded, and reuse, output age, freshness validity, and rejection causes are logged separately. Frames are independent with carried-over last-valid state rather than a full queueing simulation, which is stated so that no greater fidelity is inferred than is implemented.

B. Network Profiles and Perception Models

The five synthetic network profiles are shown in Table II. RTT is temporally correlated and outage events persist in short bursts. Local synthetic inference draws 15–35 ms latency and confidence 0.65–0.80. Edge inference draws 45–90 ms compute, confidence 0.80–0.95, and 25 kB upload per transmitted frame. Cloud inference draws 60–130 ms compute, confidence 0.82–0.96, 35 kB upload, and the additional wide-area RTT.

C. Calibration, Baselines, and Statistics

Parameters are selected on calibration seeds 0–9 only; evaluation uses disjoint seeds 100–129. A deterministic search evaluates 400 candidate configurations, including the previously published and uniform configurations. Candidates are ranked by a predeclared lexicographic objective: minimize worst-profile deadline misses; minimize stale acceptance; maximize usable confidence; minimize worst-profile p95 control latency; then minimize bandwidth. Levels within one standard error are carried forward rather than collapsed into an arbitrary

weighted score. Of the 400 candidates, 328 reach zero worst-profile deadline misses on calibration seeds, which indicates that deadline reliability follows from the policy structure rather than from a fortunate choice of weights.

The final evaluation uses 30 independent seeds and 1,000 frames per seed, profile, and policy. Eight policies are evaluated: local-only, edge-only, cloud-only, DAPPER, deadline-greedy, confidence-and-deadline, RTT-threshold, and an offline oracle. The three adaptive comparators are calibrated on the same seeds and objective as DAPPER. The oracle reads the realized outcome of the current frame and is therefore an upper bound, not a deployable policy.

The unit of statistical replication is the seed, not the frame, because frames within a run are autocorrelated. Reported intervals are 95% percentile bootstrap confidence intervals (CIs) over the 30 seed-level values using 10,000 fixed-seed resamples. Policy differences are paired seed by seed.

D. Secondary Real-Detector Validation

To replace the synthetic quality proxy with measured perception in a secondary experiment, two You Only Look Once (YOLO) family models, YOLO11n and YOLO11m [13], are evaluated on the full 5,000-image Common Objects in Context (COCO) 2017 validation set [14]. The lightweight YOLO11n acts as the local model and YOLO11m as the remote model. Detection uses confidence threshold 0.25 and intersection-over-union (IoU) matching at 0.50; recall, precision, and mean average precision (mAP) are reported. Model outputs and compute times are cached and replayed through the same policies; the network side remains synthetic. This experiment is performed on one workstation and is not an embedded or physical-robot timing study.

V. RESULTS

A. Repeated Main Benchmark

Table III reports the corrected fixed-policy comparison at $D = 100$ ms. Across all five profiles and all 30 seeds, DAPPER records a 0.00% deadline-miss rate. Local-only is also deadline-safe, so the central question is whether DAPPER recovers useful remote quality while preserving that reliability.

TABLE III
REPEATED EVALUATION OVER 30 INDEPENDENT SEEDS (1,000 FRAMES PER SEED/PROFILE/POLICY, $D = 100$ MS). CIS ARE 95% PERCENTILE BOOTSTRAP INTERVALS OVER SEEDS. CONFIDENCE IS THE SYNTHETIC PROXY HELD BY THE CONTROL LOOP AT THE DEADLINE, NOT DETECTOR ACCURACY.

Profile	Policy	p95 latency ms [95% CI]	Miss % [95% CI]	Conf. proxy	BW MB/1k fr.
Stable	local-only	34.0 [33.9,34.0]	0.00 [0.00,0.00]	0.725	0.00
	edge-only	132.8 [132.7,132.9]	48.77 [48.17,49.38]	0.448	25.00
	cloud-only	134.0 [133.9,134.0]	100.00 [100.00,100.00]	0.000	35.00
	DAPPER	34.0 [33.9,34.0]	0.00 [0.00,0.00]	0.796	21.62
Congested	local-only	33.9 [33.9,34.0]	0.00 [0.00,0.00]	0.725	0.00
	edge-only	133.9 [133.9,134.0]	100.00 [100.00,100.00]	0.000	25.00
	cloud-only	133.9 [133.9,134.0]	100.00 [100.00,100.00]	0.000	35.00
	DAPPER	33.9 [33.9,34.0]	0.00 [0.00,0.00]	0.725	0.00
Lossy	local-only	34.0 [34.0,34.1]	0.00 [0.00,0.00]	0.725	0.00
	edge-only	133.9 [133.8,133.9]	87.51 [87.06,87.94]	0.109	25.00
	cloud-only	134.0 [134.0,134.1]	100.00 [100.00,100.00]	0.000	35.00
	DAPPER	34.0 [34.0,34.1]	0.00 [0.00,0.00]	0.741	18.13
Variable	local-only	34.0 [33.9,34.0]	0.00 [0.00,0.00]	0.725	0.00
	edge-only	133.6 [133.5,133.7]	74.00 [72.62,75.43]	0.195	19.55
	cloud-only	133.7 [133.6,133.7]	78.20 [76.76,79.62]	0.158	27.37
	DAPPER	33.7 [33.7,33.8]	0.00 [0.00,0.00]	0.732	5.78
Outage	local-only	34.0 [34.0,34.1]	0.00 [0.00,0.00]	0.725	0.00
	edge-only	128.1 [127.7,128.4]	14.68 [14.06,15.33]	0.619	3.67
	cloud-only	128.1 [127.7,128.4]	14.68 [14.06,15.33]	0.619	5.14
	DAPPER	31.2 [31.0,31.4]	0.00 [0.00,0.00]	0.726	0.00

Figure 2 summarizes tail latency and the usable confidence proxy. Under the variable profile, DAPPER holds p95 control-output latency at 33.7 ms [95% CI: 33.7–33.8], compared with 133.6 ms [133.5–133.7] for edge-only and 133.7 ms [133.6–133.7] for cloud-only. Relative to local-only, DAPPER raises the usable confidence proxy under stable by 0.0710 [paired 95% CI: 0.0700–0.0720], lossy by 0.0158 [0.0151–0.0165], and variable by 0.0062 [0.0056–0.0068]. Under congested and outage profiles, the paired CI includes zero, so no quality difference is claimed.

B. Placement Behavior and Stale-Result Rejection

Table IV shows how DAPPER reaches these results and what its offloading actually buys. The policy adapts strongly to the profile: it offloads on 86.5% of stable frames but on 0.0% of outage frames, where it instead serves 73.4% of frames from a still-fresh previous output. At this deadline the committing edge-accurate mode is never selected, because the expected remote round trip does not fit the commit margin; all remote activity is hybrid, which never exposes the control loop to network delay.

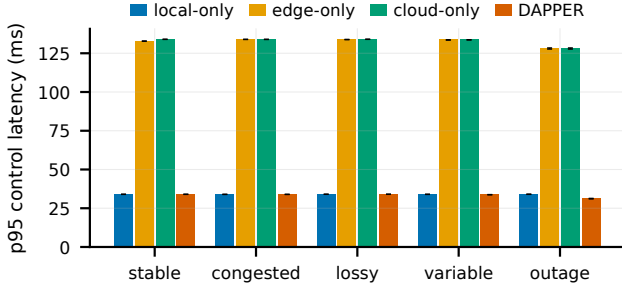
The same table quantifies stale-result rejection, which the original artifact described but did not measure. Across every policy, profile, and seed in the final evaluation—1,200 runs—the rate at which an output older than its freshness window is consumed is exactly 0.00%; a unit test asserts this end to end rather than leaving it as a claim. Rejection is nonetheless frequent: of the remote replies DAPPER requests under the stable profile, 51.2% are accepted, 47.7% are discarded for arriving after the deadline, and 1.1% are lost. Under the lossy profile only 13.7% are accepted, with 67.5% rejected on the deadline and 18.8% lost. This is also the honest explanation of the bandwidth column in Table III: because every transmitted request is charged whether or not its reply proves usable, roughly half the stable-profile traffic and the large majority of lossy-profile traffic buys nothing. The previously reported

25 kB variable-profile figure resulted from not charging lost or late attempts and is not retained.

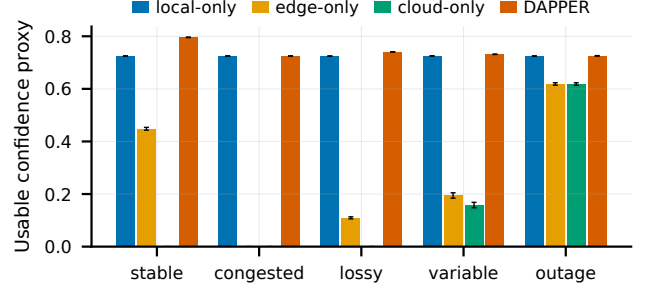
C. Adaptive Baselines, Ablation, and Sensitivity

The right half of Table IV compares DAPPER with the calibrated adaptive comparators. At $D = 100$ ms the calibrated deadline-greedy and confidence-and-deadline baselines choose settings that never transmit, making them numerically identical to local-only, and the calibrated RTT-threshold policy transmits on under 0.02% of frames. With this timing envelope, committing an appreciable share of 100-ms frames to the edge produces misses, so each baseline’s own objective drives it toward local execution. DAPPER still obtains remote quality because hybrid never commits the control loop to waiting: it keeps a local result and accepts the remote reply only if it actually arrives on time. Against the non-deployable offline oracle, DAPPER recovers approximately 92% of the oracle’s available gain over local-only under the stable profile (0.7250 local-only, 0.7960 DAPPER, 0.8018 oracle), although it uses more bandwidth (21.62 versus 12.81 MB/1,000 frames), exposing room for better runtime prediction.

Table V reports one-at-a-time ablations. Removing packet loss from the risk score is the most consequential single change: it raises the risk of the remaining signals after renormalization, suppresses useful offloading, and reduces stable confidence from 0.7960 to 0.7675 and lossy confidence from 0.7406 to 0.7248. Removing the freshness gate raises variable-profile traffic from 5.78 to 9.75 MB/1,000 frames (about 69%) without a statistically distinguishable confidence gain, which is the clearest demonstration that the gate is a bandwidth mechanism rather than a deadline mechanism. Disabling the confidence gate raises both confidence and traffic, confirming that this gate is the quality/bandwidth knob. Disabling degraded-safe reuse leaves the miss rate at zero but increases outage p95 latency from 31.19 to 33.85 ms. Removing frame age changes no reported metric, consistent



(a) p95 control-output latency.



(b) usable confidence proxy at the deadline.

Fig. 2. Repeated synthetic benchmark at $D = 100$ ms. Error bars are 95% bootstrap CIs over 30 independent seeds. The confidence proxy is not detector accuracy.

TABLE IV

LEFT: DAPPER PLACEMENT AND REMOTE ACCOUNTING AT $D = 100$ MS OVER 30 SEEDS; ACCEPTED IS A PERCENTAGE OF TRANSMITTED REQUESTS, REUSE AND STALE OF ALL FRAMES. RIGHT: DAPPER AGAINST THE CALIBRATED ADAPTIVE COMPARATORS AND THE OFFLINE ORACLE, AVERAGED OVER THE FIVE PROFILES.

Profile	Mode share %				Remote	Reuse	Stale
	local	edge	hybrid	degr.	acc. %	%	%
Stable	13.5	0.0	86.5	0.0	51.2	0.0	0.00
Congested	99.2	0.0	0.0	0.7	0.0	0.7	0.00
Lossy	27.5	0.0	72.5	0.0	13.7	0.0	0.00
Variable	60.1	0.0	23.1	16.8	15.9	16.8	0.00
Outage	22.6	0.0	0.0	77.4	0.0	73.4	0.00

Policy	Tx %	Miss %	p95	Conf.	MB/1k
local-only	0.00	0.00	34.0	0.7250	0.00
deadline-greedy	0.00	0.00	34.0	0.7250	0.00
conf.+deadline	0.00	0.00	34.0	0.7250	0.00
RTT-threshold	0.01	0.00	34.0	0.7250	0.00
DAPPER	36.42	0.00	33.4	0.7437	9.11
oracle [†]	13.58	0.00	60.8	0.7454	3.40

[†]Offline upper bound: reads the realized frame outcome; not deployable.

TABLE V

ONE-AT-A-TIME ABLATION, AVERAGED OVER THE FIVE PROFILES AND 30 SEEDS. REMOVING A RISK SIGNAL SETS ITS WEIGHT TO ZERO AND RENORMALIZES THE REMAINDER.

Variant	Miss %	p95 ms	Conf. proxy	MB/1k fr.
full DAPPER	0.00	33.37	0.7437	9.11
– RTT	0.00	33.43	0.7427	8.09
– packet loss	0.00	32.58	0.7337	2.50
– edge load	0.00	33.37	0.7431	8.14
– frame age	0.00	33.37	0.7437	9.11
– deadline pressure	0.00	33.38	0.7439	9.71
– confidence gate	0.00	33.37	0.7453	10.53
– freshness gate	0.00	33.37	0.7437	9.96
– degraded reuse	0.00	33.95	0.7436	9.11

with its zero calibrated weight. No ablated variant produced a nonzero deadline-miss rate.

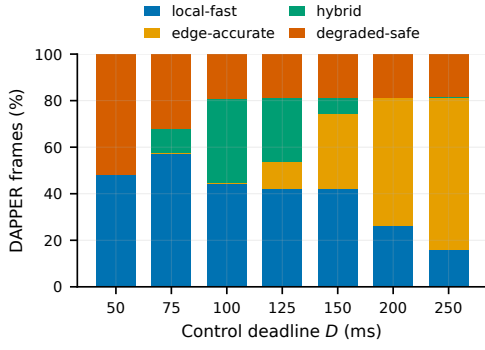
Figure 3 shows two complementary sensitivity results. Edge-accurate is not dead code: it receives no frames at $D \leq 100$ ms, first appears at 125 ms, and reaches 65.5% of decisions at 250 ms. However, the frozen single-deadline calibration is not reliable at every larger deadline: the miss rate is 5.24%, 13.31%, and 4.21% at 125, 150, and 200 ms, respectively. The commit margin m is unidentifiable at the 100-ms calibration point because no frame enters edge-accurate there. A direct sweep of that parameter over evaluation seeds confirms the mechanism and bounds it: any $m \leq 0.6$ holds the miss rate at 0.00% at every swept deadline from 100 to 200 ms, whereas the calibrated $m = 0.8$ does not. We report this diagnostic but deliberately do not adopt its value, because it was obtained on evaluation rather than calibration data; multi-deadline calibration is the correct remedy.

Runtime-estimation error is strongly asymmetric. Biasing RTT estimates anywhere from -30% to $+30\%$ leaves the miss rate at 0.00% in the tested stable, congested, and variable profiles. Underestimating edge compute is the dangerous direction: a 10% underestimate produces a worst-profile miss rate of 0.02%, while a 30% underestimate reaches 38.55%; jointly underestimating RTT and compute by 30% reaches 42.22%. Overestimation makes the scheduler more conservative and does not create misses over the tested range. The mechanism is the same one the deadline sweep exposes: an underestimated completion passes the commit test, and a committed frame retains no local answer.

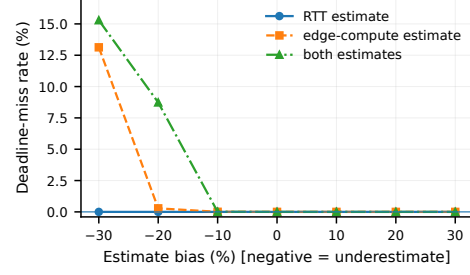
D. Scheduling Overhead and Real-Detector Validation

Over 200,000 timed calls after warm-up, one DAPPER decision takes $1.45 \mu\text{s}$ on average (median 1.40, p95 1.80, p99 $2.00 \mu\text{s}$) in CPython on the evaluation workstation, and running DAPPER instead of local-only adds $1.31 \mu\text{s}$ per frame to benchmark wall time. These measure software overhead on a desktop processor, not robot hardware or application inference latency. Depending on profile, DAPPER changes mode 12–418 times per 1,000 frames; no physical switching cost is modeled.

Table VI shows the measured detector gap. YOLO11m increases overall recall by 13.1 percentage points and recall on the selected safety-relevant classes (person, bicycle, car, motorcycle, bus, truck) by 11.1 points over YOLO11n. Mean inference times are 35.2 ms (YOLO11n) and 177.5 ms (YOLO11m) on the workstation central processing unit (CPU), and 6.1 ms and 8.1 ms on its graphics processing unit (GPU).



(a) DAPPER mode share versus control deadline.



(b) Profile-mean deadline misses versus scheduler estimate bias.

Fig. 3. Sensitivity analysis. Panel (a) is pooled over five profiles and 30 seeds. Panel (b) averages the three profiles used in the estimation-error study; negative bias means the scheduler underestimates the true runtime.

TABLE VI
REAL-DETECTOR QUALITY ON ALL 5,000 COCO VAL2017 IMAGES.

Model	Recall	Safety recall	mAP50	mAP50-95
YOLO11n (local)	0.507	0.623	0.546	0.389
YOLO11m (remote)	0.638	0.734	0.678	0.510

Table VII replays every policy over these measured outputs. Under the stable profile DAPPER delivers 0.674 recall against 0.639 for local-only, with 0.00% deadline misses for both, and 0.762 versus 0.737 on safety-relevant classes; edge-only reaches higher recall but misses deadlines. Under the lossy profile the ordering is unambiguous: DAPPER delivers more recall than either fixed policy *and* misses no deadlines, whereas edge-only misses 19.74%. Under variable and outage conditions DAPPER’s all-frame figure falls below local-only because 16.0% and 65.0% of its frames respectively reuse a previous output, and a reused output is scored as detecting nothing in the current image: consecutive COCO validation images are unrelated, so this is a lower bound that a real video stream would not incur. The fresh-frame column, which averages only over frames that computed a new output, shows no such gap. Both views are reported rather than choosing whichever is more favorable.

VI. DISCUSSION AND THREATS TO VALIDITY

The results support a deliberately narrow conclusion. At the primary 100-ms operating point, DAPPER preserves the deadline behavior of local inference while recovering part of the quality available from a stronger remote model when the network can support a timely refinement. Its structural advantage over the calibrated adaptive comparators is the hybrid path: the system can ask for a better answer without forcing the control loop to wait for one. The key observations are:

- **Deadline reliability is structural, not tuned.** 328 of 400 candidate configurations reach zero worst-profile misses, and no ablated variant produced a miss.

TABLE VII
PER-FRAME DETECTION QUALITY DELIVERED WHEN THE MEASURED DETECTOR OUTPUTS ARE REPLAYED UNDER THE SYNTHETIC PROFILES ($D = 100$ MS, 10 SEEDS). “ALL” SCORES A REUSED OUTPUT AS ZERO; “FRESH” AVERAGES ONLY FRESHLY COMPUTED OUTPUTS.

Profile	Policy	Delivered recall		Miss	Reuse
		all	fresh	%	%
Stable	local-only	0.639	0.639	0.00	0.0
	edge-only	0.740	0.747	0.96	0.0
	DAPPER	0.674	0.674	0.00	0.0
Lossy	local-only	0.639	0.639	0.00	0.0
	edge-only	0.600	0.747	19.74	0.0
	DAPPER	0.663	0.663	0.00	0.0
Variable	local-only	0.639	0.639	0.00	0.0
	edge-only	0.680	0.721	5.66	0.0
	DAPPER	0.545	0.649	0.00	16.0
Outage	local-only	0.639	0.639	0.00	0.0
	edge-only	0.546	0.639	14.54	0.0
	DAPPER	0.223	0.637	0.00	65.0

- **The gain over local-only is real but modest and profile-dependent**, and it is absent where the network cannot deliver a timely refinement.
- **Bandwidth is the price.** Once lost and late attempts are charged, about half of stable-profile traffic and most lossy-profile traffic buys no usable result.
- **Risk is concentrated in committing.** Both the deadline sweep and the estimation-error sweep fail through the same branch; hybrid is never implicated.
- **Freshness gating is a bandwidth mechanism**, saving roughly 69% of variable-profile traffic at no measurable quality cost.

Several limitations restrict generalization. The main benchmark uses parameterized synthetic profiles rather than measured Wi-Fi, fifth-generation (5G), or private-edge traces; trace replay is implemented and tested, but no measured trace was collected, so no result rests on one. The synthetic confidence metric is a proxy, not detector accuracy; the secondary detector experiment removes that limitation but still replays synthetic networking, with both models measured on one workstation rather than embedded hardware. The timing model does not simulate queue buildup between frames. Energy is not instrumented, physical switching cost is not modeled, and no hazard

analysis or formal safety guarantee is provided—*degraded-safe* denotes conservative fallback behavior only. Finally, parameters are calibrated at $D = 100$ ms, and the larger-deadline results show that the frozen configuration should not be assumed valid for all control budgets.

Internal validity rests on exact paired scenarios, one shared execution model, disjoint calibration/evaluation seeds, and seed-level intervals. Every figure and table here is regenerated from released result files by artifact scripts, and the full protocol runs from a single command. Future work should add physical-robot experiments, measured wireless traces, embedded power measurement, queue-aware timing, and multi-deadline calibration.

VII. CONCLUSION

This paper presented DAPPER, a deterministic deadline- and confidence-aware perception-placement framework for robotic edge computing. The revised evaluation spans 1.2 million frame-level decisions over five network profiles, eight policies, and 30 independent seeds. At the calibrated 100-ms deadline, DAPPER records 0.00% deadline misses under all evaluated synthetic profiles while holding tail latency at local-inference levels, never accepting an output beyond its freshness window, and improving the usable confidence proxy over local-only when timely remote refinement is possible. Ablation and sensitivity studies identify packet-loss awareness, freshness gating, and conservative remote commitment as the decisive design factors, and locate the architecture’s failure mode in the decision to commit a frame to remote execution. A secondary experiment with YOLO11n and YOLO11m on COCO val2017 confirms a real perception-quality gap and shows stable/lossy recall gains under DAPPER replay without deadline misses. These findings support deadline-aware, freshness-gated hybrid placement as a useful mechanism, but they do not establish real-world robotic safety. Physical

validation, measured network traces, energy instrumentation, and calibration across control deadlines remain necessary next steps.

REFERENCES

- [1] J. Ichnowski *et al.*, “FogROS2: An adaptive platform for cloud and fog robotics using ROS 2,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2023, pp. 5006–5013.
- [2] K. Chen, N. Tian, C. Juette, T. Qiu, L. Ren, J. Kubiawicz, and K. Goldberg, “FogROS2-PLR: Probabilistic latency-reliability for cloud robotics,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2025, pp. 16290–16297, doi: 10.1109/ICRA55743.2025.11127588.
- [3] B. Liu *et al.*, “ElasticROS: An elastically collaborative robot operating system for fog and cloud robotics,” arXiv:2209.01774, 2022.
- [4] X. Yang and B. Hu, “An adaptive ROS2 node deployment framework in mobile edge-robot systems,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2025, pp. 12652–12658, doi: 10.1109/IROS60139.2025.11247728.
- [5] M.-D. Pavel, M. De Marchi, and G. Stamatescu, “Edge-cloud offloading for real-time perception of resource-constrained AMR in IIoT environments,” in *Proc. 22nd Int. Conf. Distributed Computing in Smart Systems and the Internet of Things (DCOSS-IoT)*, 2026, pp. 521–528, doi: 10.1109/DCOSS-IoT69657.2026.00087.
- [7] E. Li, Z. Zhou, and X. Chen, “Edge intelligence: On-demand deep learning model co-inference with device-edge synergy,” in *Proc. ACM SIGCOMM Workshop Mobile Edge Commun.*, 2018, pp. 31–36.
- [8] S. Raj, R. Mittal, H. Gupta, and Y. Simmhan, “Adaptive heuristics for scheduling DNN inferencing on edge and cloud for personalized UAV fleets,” *Future Generation Computer Systems*, vol. 173, Art. no. 107874, 2025, doi: 10.1016/j.future.2025.107874.
- [9] M. Goudarzi, M. Palaniswami, and R. Buyya, “Scheduling IoT applications in edge and fog computing environments: A taxonomy and future directions,” *ACM Computing Surveys*, vol. 55, no. 7, pp. 1–41, 2023.
- [10] S. Iftikhar *et al.*, “AI-based fog and edge computing: A systematic review, taxonomy and future directions,” *Internet of Things*, vol. 21, Art. no. 100674, 2023.
- [11] W. Qian, R. W. L. Coutinho, and A. Boukerche, “A novel DRL-based orchestrator for edge computing resource allocation in mobile computer vision systems,” *Computer Networks*, vol. 285, Art. no. 112380, 2026, doi: 10.1016/j.comnet.2026.112380.
- [12] Open Robotics, “About quality of service settings,” *ROS 2 Documentation*. [Online]. Available: <https://docs.ros.org/en/rolling/Concepts/Intermediate/About-Quality-of-Service-Settings.html>
- [13] Ultralytics, “Ultralytics YOLO11,” 2024. [Online]. Available: <https://docs.ultralytics.com/models/yolo11/>
- [14] T.-Y. Lin *et al.*, “Microsoft COCO: Common objects in context,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2014, pp. 740–755.
- [6] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, “Neurosurgeon: Collaborative intelligence between the cloud and mobile edge,” in *Proc. ASPLOS*, 2017, pp. 615–629.